

Kafka - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

KAFKA — MASTER NOTES	5
A Complete Reference Book for Interviews, Production Work & System Understanding	5
TOPIC 1: BROKERS	5
1. Introduction	5
1.1 What is it, and why?	5
1.2 What is a Broker?	5
2. Redis Pub/Sub vs Kafka — The Critical Distinction (Recap + Expansion)	5
3. FULL STEPS: Running Kafka Locally via Docker	6
STEP 1 — Create a docker-compose.yml for Kafka	6
STEP 2 — Start It	6
STEP 3 — Verify With Kafka's Command-Line Tools (Inside the Container)	6
4. Real Life Analogy	6
5. Frequently Asked Interview Questions	6
TOPIC 2: TOPICS	7
1. Introduction	7
2. Code Example	7
3. Real Life Analogy	7
4. Frequently Asked Interview Questions	7
TOPIC 3: PARTITIONS	7
1. Introduction	7
2. How Messages Are Assigned to Partitions	8
3. Diagram	8
4. Real Life Analogy	8
5. Edge Cases	8
6. Common Mistakes	8
7. Frequently Asked Interview Questions	8
TOPIC 4: PRODUCERS	9

1. Introduction	9
2. Code Example — Plain Java Kafka Producer	9
3. Delivery Guarantees — acks Configuration	10
4. Real Life Analogy	10
5. Frequently Asked Interview Questions	10

TOPIC 5: CONSUMERS **10**

1. Introduction	10
2. Code Example — Plain Java Kafka Consumer	11
3. Real Life Analogy	11
4. Frequently Asked Interview Questions	11

TOPIC 6: CONSUMER GROUPS **11**

1. Introduction	12
2. How Partition Assignment Works	12
3. Multiple Independent Consumer Groups — Broadcasting	12
4. Real Life Analogy	12
5. Edge Cases	12
6. Frequently Asked Interview Questions	12

TOPIC 7: OFFSETS **13**

1. Introduction	13
2. Diagram	13
3. Committing Offsets — Automatic vs Manual	13
4. Real Life Analogy	14
5. Frequently Asked Interview Questions	14

TOPIC 8: SPRING KAFKA INTEGRATION **14**

1. Introduction	14
2. FULL STEP-BY-STEP Setup	14
STEP 1 — Add the Dependency	14
STEP 2 — Configure	15
STEP 3 — Producing Messages (KafkaTemplate)	15
STEP 4 — Consuming Messages (@KafkaListener)	15

3. Real Life Analogy 15

4. Common Mistakes 15

5. Frequently Asked Interview Questions 16

6. Revision Notes (Entire Kafka Section Consolidated) 16

7. Homework (Covers Entire Kafka Section) 16

FINAL SUMMARY — KAFKA COMPLETE 17

KAFKA — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: BROKERS

1. Introduction

1.1 What is it, and why?

Apache Kafka is a **distributed event streaming platform** — originally built at LinkedIn, designed to handle MASSIVE volumes of real-time data (millions of events per second) with DURABLE, PERSISTENT storage and GUARANTEED delivery — directly solving the exact limitation Redis Pub/Sub (previous section) explicitly does NOT: messages are written to DISK and RETAINED, so a consumer that's offline when a message is sent can still read it LATER, and messages can be REPLAYED.

1.2 What is a Broker?

A **broker** is a single Kafka SERVER — it stores data and serves client requests (producers writing data, consumers reading it). A Kafka **cluster** is made up of MULTIPLE brokers working together, providing both SCALABILITY (spreading load across machines) and FAULT TOLERANCE (if one broker fails, others continue serving).

2. Redis Pub/Sub vs Kafka — The Critical Distinction (Recap + Expansion)

Aspect	Redis Pub/Sub	Kafka
Message persistence	NONE — fire and forget	Messages are WRITTEN TO DISK and RETAINED (for a configurable period)
Offline subscriber	MISSES messages entirely	Can catch up later — reads from wherever it LAST left off
Replay messages	Not possible	Fully possible — consumers can re-read from any point in history
Throughput at scale	Good for lightweight use cases	Built specifically for MASSIVE, sustained, high-throughput event streams
Typical use case	Simple real-time notifications	Event-driven microservices, log aggregation, real-time analytics pipelines, audit trails

3. FULL STEPS: Running Kafka Locally via Docker

STEP 1 — Create a `docker-compose.yml` for Kafka

```
version: "3.8"
services:
  kafka:
    image: apache/kafka:3.7.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
```

STEP 2 — Start It

```
docker compose up -d
```

STEP 3 — Verify With Kafka's Command-Line Tools (Inside the Container)

```
docker exec -it <container-id> /bin/bash

kafka-topics.sh --create --topic orders --bootstrap-server localhost:9092
kafka-topics.sh --list --bootstrap-server localhost:9092
kafka-console-producer.sh --topic orders --bootstrap-server localhost:9092
kafka-console-consumer.sh --topic orders --from-beginning --bootstrap-server localhost:9092
```

4. Real Life Analogy

- **Bank:** A Kafka broker is like ONE physical bank branch that STORES and MANAGES the actual PERMANENT transaction ledger — a whole Kafka cluster is like the bank's ENTIRE NETWORK of branches, working together so the bank keeps functioning even if one specific branch temporarily goes offline, and every transaction is permanently recorded, never simply "missed" if you weren't watching at that exact moment.

5. Frequently Asked Interview Questions

- 1 **What is Apache Kafka?** A distributed event streaming platform providing durable, persistent, high-throughput messaging with guaranteed delivery and replay capability.
- 2 **What is a Kafka broker?** A single Kafka server that stores data and serves producer/consumer requests; multiple brokers form a cluster.
- 3 **What's the key difference between Kafka and Redis Pub/Sub?** Kafka persists messages to disk and retains them, allowing offline consumers to catch up and messages to be replayed; Redis Pub/Sub is fire-and-forget with no persistence at all.
- 4 **Why would a company choose Kafka over Redis Pub/Sub for critical events?** Kafka guarantees no message is lost even if a consumer is temporarily offline, and supports massive sustained throughput — essential for critical business events (orders, payments) versus Redis's lightweight, best-effort notifications.

TOPIC 2: TOPICS

1. Introduction

What is it? A **topic** is a named CATEGORY/FEED to which messages are published — conceptually similar to a database table, or a specific "channel" (though far more powerful/durable than Redis's Pub/Sub channels). Producers WRITE to topics; consumers READ from them.

2. Code Example

```
kafka-topics.sh --create --topic order-events --bootstrap-server localhost:9092 --partitions 3
--replication-factor 1
kafka-topics.sh --describe --topic order-events --bootstrap-server localhost:9092
kafka-topics.sh --delete --topic order-events --bootstrap-server localhost:9092
```

3. Real Life Analogy

- **Restaurant:** A topic is like a SPECIFIC, dedicated bulletin board in the kitchen — "New Orders," "Ready for Pickup," "Inventory Alerts" — each board holds ONLY its own category of messages, and different staff members (consumers) watch whichever specific board is relevant to their role.

4. Frequently Asked Interview Questions

- 1 **What is a Kafka topic?** A named category/feed to which producers publish messages and from which consumers read them.
- 2 **Can multiple producers write to the same topic, and multiple consumers read from it?** Yes — topics support many producers and many consumers simultaneously.

TOPIC 3: PARTITIONS

1. Introduction

What is it? A topic is SPLIT into one or more **partitions** — the actual unit of PARALLELISM and SCALABILITY in Kafka. Each partition is an ORDERED, IMMUTABLE sequence of messages (a log), and DIFFERENT partitions of the SAME topic can be spread across DIFFERENT brokers, allowing a topic to handle far more throughput than a single machine could alone.

2. How Messages Are Assigned to Partitions

```
Producer sends a message with a KEY (e.g., "order-101")
|
v
Kafka HASHES the key -> determines WHICH partition this message goes to
|
v
ALL messages with the SAME key ALWAYS go to the SAME partition
-> guarantees ORDERING for that specific key (e.g., all events for "order-101"
are processed in the exact order they were sent)

Messages with NO key are distributed round-robin across partitions (no ordering
guarantee across the whole topic, only within each individual partition)
```

3. Diagram

```
Topic: "order-events" (3 partitions)

Partition 0: [msg1]-[msg4]-[msg7]-[msg10]... <- ordered sequence within THIS partition
Partition 1: [msg2]-[msg5]-[msg8]-[msg11]...
Partition 2: [msg3]-[msg6]-[msg9]-[msg12]...

Kafka guarantees ORDER *within* a single partition, NOT across the whole topic!
```

4. Real Life Analogy

- **Bank:** Partitions are like a bank splitting its overall transaction log into MULTIPLE separate ledger books (one per region, say), each handled independently and in parallel by different clerks — but EVERY transaction for a SPECIFIC customer always goes into THEIR SAME assigned ledger book, guaranteeing that customer's OWN transaction history stays correctly ordered, even though different customers' books are processed simultaneously and independently.

5. Edge Cases

Case	Result
A topic with only ONE partition	No parallelism possible for that topic — all messages processed strictly sequentially by a single consumer at a time
Messages sent WITHOUT a key	Distributed round-robin — no cross-message ordering guarantee at all

6. Common Mistakes

- Assuming Kafka guarantees GLOBAL ordering across an entire topic — it only guarantees ordering WITHIN a single partition; use a consistent message KEY when per-entity ordering matters (e.g., always keying by `orderId` so all events for that specific order stay in order).
- Under-provisioning partitions for a high-throughput topic, limiting how much you can parallelize consumption later (partition count can be increased, but never decreased, on an existing topic).

7. Frequently Asked Interview Questions

- 1 **What is a Kafka partition?** A subdivision of a topic — an ordered, immutable log of messages, and the fundamental unit of parallelism/scalability in Kafka.
 - 2 **Does Kafka guarantee ordering across an ENTIRE topic?** No — only WITHIN a single partition. Global topic-wide ordering is not guaranteed unless the topic has just one partition.
 - 3 **How does Kafka decide which partition a message goes to?** By hashing the message's KEY (if provided) — messages with the same key always land in the same partition, guaranteeing per-key ordering; keyless messages are distributed round-robin.
 - 4 **Why might you choose a message key deliberately (e.g., always the `orderId`)?** To guarantee all events related to that SAME entity are processed in the correct order, since they'll always land in the same partition.
-
-

TOPIC 4: PRODUCERS

1. Introduction

What is it? A producer is any client application that WRITES (publishes) messages to a Kafka topic.

2. Code Example — Plain Java Kafka Producer

```
import org.apache.kafka.clients.producer.*;
import java.util.Properties;

public class OrderProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

        try (KafkaProducer<String, String> producer = new KafkaProducer<>(props)) {
            ProducerRecord<String, String> record =
                new ProducerRecord<>("order-events", "order-101", "Order placed: 2 items, $150");

            producer.send(record, (metadata, exception) -> {
                if (exception == null) {
                    System.out.println("Sent to partition " + metadata.partition() + ", offset " + metadata.offset());
                } else {
                    exception.printStackTrace();
                }
            });
        }
    }
}
```

3. Delivery Guarantees — `acks` Configuration

<code>acks</code> Value	Guarantee	Trade-off
<code>acks=0</code>	Producer doesn't wait for ANY confirmation	Fastest, but messages can be SILENTLY LOST
<code>acks=1</code>	Waits for the LEADER broker to acknowledge	Reasonable balance — can still lose data if the leader crashes before replicating
<code>acks=all</code> (or <code>-1</code>)	Waits for ALL in-sync replicas to acknowledge	Strongest durability guarantee, at the cost of some latency

4. Real Life Analogy

- **Restaurant:** A producer is like a waiter WRITING a new order onto the kitchen's official order rail — `acks=all` is like requiring EVERY relevant kitchen station to confirm they've seen the order before the waiter moves on (safest, slightly slower); `acks=0` is like the waiter just pinning it up and walking away immediately, without confirming ANYONE actually saw it (fastest, but risks the order being missed entirely).

5. Frequently Asked Interview Questions

- 1 **What is a Kafka producer?** A client application that publishes (writes) messages to a Kafka topic.
- 2 **What does the `acks` producer configuration control?** How many broker acknowledgments the producer waits for before considering a message successfully sent — trading off speed against durability guarantees.
- 3 **Which `acks` setting provides the strongest durability guarantee?** `acks=all` (waits for all in-sync replicas to acknowledge).

TOPIC 5: CONSUMERS

1. Introduction

What is it? A consumer is a client application that READS messages from a Kafka topic, tracking its own position (Topic 7: Offsets) so it knows exactly where it left off.

2. Code Example — Plain Java Kafka Consumer

```
import org.apache.kafka.clients.consumer.*;
import java.time.Duration;
import java.util.*;

public class OrderConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "order-processing-service"); // Consumer Group (Topic 6!)
        props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("auto.offset.reset", "earliest"); // where to start if no offset is stored yet

        try (KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props)) {
            consumer.subscribe(List.of("order-events"));

            while (true) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));
                for (ConsumerRecord<String, String> record : records) {
                    System.out.printf("Received: key=%s, value=%s, partition=%d, offset=%d%n",
                        record.key(), record.value(), record.partition(), record.offset());
                }
            }
        }
    }
}
```

3. Real Life Analogy

- **Restaurant:** A consumer is like a specific kitchen station STAFF MEMBER continuously WATCHING the order rail, picking up and processing new tickets as they appear, remembering exactly which ticket number they last handled so they never accidentally re-process or skip one.

4. Frequently Asked Interview Questions

- 1 **What is a Kafka consumer?** A client application that reads (subscribes to and processes) messages from a Kafka topic.
- 2 **What does `auto.offset.reset=earliest` mean?** If the consumer has no previously stored offset (e.g., it's brand new), start reading from the VERY BEGINNING of the topic's retained history, rather than only new messages from now on.

TOPIC 6: CONSUMER GROUPS

1. Introduction

What is it? A consumer group is a set of consumers that COOPERATE to consume a topic's partitions TOGETHER — Kafka automatically DIVIDES the topic's partitions AMONG the group's members, so each message is processed by EXACTLY ONE consumer WITHIN that group, enabling parallel processing at scale.

2. How Partition Assignment Works

Topic "order-events" has 3 PARTITIONS.
Consumer Group "order-processing-service" has 3 CONSUMER instances (e.g., 3 running app pods).

Kafka automatically assigns:
Consumer-1 -> Partition 0
Consumer-2 -> Partition 1
Consumer-3 -> Partition 2

Each message is processed by EXACTLY ONE consumer in the group - this is how you SCALE OUT processing: add more consumer instances (up to the partition count) to process more messages in PARALLEL.

3. Multiple Independent Consumer Groups — Broadcasting

Topic "order-events"
|
+-- Consumer Group "email-service" -> processes EVERY message (sends confirmation emails)
+-- Consumer Group "analytics-service" -> ALSO processes EVERY message (independently, for reporting)
+-- Consumer Group "inventory-service" -> ALSO processes EVERY message (independently, updates stock)

Each SEPARATE consumer group receives its OWN independent copy of EVERY message -
this is how Kafka supports "PUBLISH ONCE, MULTIPLE INDEPENDENT SYSTEMS REACT" patterns.

4. Real Life Analogy

- **Restaurant:** A consumer group is like a TEAM of kitchen staff assigned to jointly work through a SINGLE order rail — Kafka automatically splits the incoming tickets among them so no ticket is handled TWICE within that team, while ANOTHER, entirely separate team (a different consumer group — e.g., the delivery-dispatch team) can ALSO watch the SAME rail independently, each team getting its OWN full view of every order.

5. Edge Cases

Case	Result
MORE consumers in a group than the topic has partitions	The EXTRA consumers sit IDLE — a partition can only be actively consumed by ONE consumer within a group at a time
A consumer in a group CRASHES	Kafka automatically REBALANCES, reassigning that consumer's partitions to the REMAINING active consumers in the group

6. Frequently Asked Interview Questions

- 1 **What is a Kafka consumer group?** A set of consumers that cooperate to consume a topic's messages together, with Kafka automatically dividing partitions among them so each message is processed exactly once within the group.
- 2 **How do you scale out message processing in Kafka?** Add more consumer instances to a consumer group, up to the number of partitions the topic has.
- 3 **What happens if you have MORE consumers in a group than partitions?** The extra consumers remain idle — a partition is only actively assigned to one consumer within a group at a time.
- 4 **How can multiple independent services each process EVERY message from the same topic?** Each service runs its OWN separate consumer group — Kafka delivers a full, independent copy of every message to each distinct group.

TOPIC 7: OFFSETS

1. Introduction

What is it? An offset is a sequential, per-PARTITION number identifying EXACTLY where a consumer is in that partition's message log — Kafka tracks (COMMITs) each consumer group's current offset per partition, so consumers can resume EXACTLY where they left off after a restart, rather than reprocessing everything or missing messages.

2. Diagram

```
Partition 0: [offset 0] [offset 1] [offset 2] [offset 3] [offset 4] ...
^
Consumer's CURRENT position
(committed offset = 2, meaning
offsets 0,1,2 have been processed;
next poll starts from offset 3)
```

3. Committing Offsets — Automatic vs Manual

```
props.put("enable.auto.commit", "true"); // Kafka periodically auto-commits offsets (simpler, less
precise)
props.put("auto.commit.interval.ms", "5000");

// OR, manual commit for PRECISE control (commit ONLY after successfully processing):
while (true) {
    ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));
    for (ConsumerRecord<String, String> record : records) {
        processOrder(record.value()); // your actual business logic
    }
    consumer.commitSync(); // ONLY commit AFTER successful processing
}
```

Why manual commit often matters: with AUTO-commit, if your consumer CRASHES after Kafka auto-commits an offset but BEFORE your application actually finished processing that message, the message is EFFECTIVELY LOST (Kafka thinks it was handled). Manual commit — committing ONLY after

your business logic genuinely succeeds — avoids this, trading a little complexity for a stronger "at-least-once" processing guarantee.

4. Real Life Analogy

- **Restaurant:** An offset is like a BOOKMARK a kitchen staff member keeps in their specific order rail, marking exactly "I've completed up through order #47" — if they step away and return later (or a NEW staff member takes over their station), they know EXACTLY where to resume, without redoing already-completed work or accidentally skipping any orders.

5. Frequently Asked Interview Questions

- 1 **What is a Kafka offset?** A sequential, per-partition identifier marking a consumer's exact position within that partition's message log.
 - 2 **Why does manual offset commit provide a stronger guarantee than auto-commit?** Because it commits ONLY after your application has genuinely finished processing a message — auto-commit can mark a message as "handled" even if your application crashes mid-processing, effectively losing it.
 - 3 **Can a consumer resume from a specific earlier offset?** Yes — Kafka retains messages for a configurable retention period, allowing a consumer to seek back to and reprocess/replay any earlier offset within that window.
-

TOPIC 8: SPRING KAFKA INTEGRATION

1. Introduction

What is it? `spring-kafka` provides `@KafkaListener` and `KafkaTemplate`, dramatically simplifying Kafka producer/consumer code compared to the raw Java client APIs shown in Topics 4-5 — following the SAME pattern Spring Boot uses throughout this course (annotations replacing verbose boilerplate).

2. FULL STEP-BY-STEP Setup

STEP 1 — Add the Dependency

```
<dependency>
<groupId>org.springframework.kafka</groupId>
<artifactId>spring-kafka</artifactId>
</dependency>
```

STEP 2 — Configure

```
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=order-processing-service
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.StringSerializer
```

STEP 3 — Producing Messages (`KafkaTemplate`)

```
@Service
public class OrderEventPublisher {
    private final KafkaTemplate<String, String> kafkaTemplate;

    public OrderEventPublisher(KafkaTemplate<String, String> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publishOrderPlaced(String orderId, String details) {
        kafkaTemplate.send("order-events", orderId, details); // topic, key, value - ONE line!
    }
}
```

STEP 4 — Consuming Messages (`@KafkaListener`)

```
@Component
public class OrderEventListener {

    @KafkaListener(topics = "order-events", groupId = "order-processing-service")
    public void handleOrderEvent(ConsumerRecord<String, String> record) {
        System.out.println("Received order event: key=" + record.key() + ", value=" + record.value());
        // ... actual business logic: update inventory, send confirmation email, etc.
    }
}
```

Notice: NO manual `poll()` loop, NO manual `KafkaConsumer` setup — `@KafkaListener` handles the ENTIRE consumption loop internally, calling your method for EACH received message, exactly like `@RabbitListener` or any other Spring messaging abstraction follows the SAME "annotate a method, Spring handles the plumbing" philosophy established throughout Spring Boot.

3. Real Life Analogy

- **Restaurant:** `@KafkaListener` is like having a fully-automated kitchen NOTIFICATION SYSTEM that DIRECTLY calls a specific staff member's name and hands them the ticket the MOMENT it arrives — no one needs to manually stand and continuously check the rail themselves; the system handles all that watching/polling automatically.

4. Common Mistakes

- Forgetting `group-id` configuration, causing consumers to unintentionally form SEPARATE, unexpected consumer groups.
- Not handling deserialization exceptions gracefully (a malformed message can otherwise repeatedly crash the listener).

5. Frequently Asked Interview Questions

- 1 **What does `@KafkaListener` do?** Automatically handles the entire Kafka consumption loop (polling, deserializing, dispatching), calling your annotated method for each received message — eliminating manual `KafkaConsumer` boilerplate.
- 2 **What does `KafkaTemplate.send()` do?** Publishes a message to a specified topic (with an optional key), abstracting away the raw `KafkaProducer` API.
- 3 **How is Spring Kafka's philosophy consistent with the rest of Spring Boot covered in this course?** It follows the same "annotate a method, let Spring handle the underlying plumbing" pattern used throughout — `@Scheduled`, `@Async`, `@Transactional`, `@Cacheable`, and now `@KafkaListener`.

6. Revision Notes (Entire Kafka Section Consolidated)

KAFKA – CHEAT SHEET

=====

SOLVES: Redis Pub/Sub's fire-and-forget limitation - Kafka PERSISTS messages to disk, supports REPLAY, offline consumers can catch up later

BROKER: one Kafka server; **CLUSTER** = multiple brokers (scale + fault tolerance)
TOPIC: named category/feed producers publish to, consumers read from
PARTITION: a topic's subdivisions - the unit of PARALLELISM
ordering GUARANTEED only WITHIN a partition, not across the whole topic
same KEY -> always same partition (guarantees per-entity ordering)

PRODUCER: writes messages. `acks=0` (fastest, riskiest) / `acks=1` / `acks=all` (safest, waits for all in-sync replicas)
CONSUMER: reads messages, tracks its OWN position (offset)

CONSUMER GROUP: multiple consumers COOPERATING on ONE topic - Kafka splits partitions among them, each message processed ONCE per group
MULTIPLE separate consumer groups each get their OWN full copy of every message (broadcast pattern)
more consumers than partitions -> extras sit IDLE
consumer crash -> automatic REBALANCE to remaining consumers

OFFSET: per-partition position marker. auto-commit (simpler, can lose messages on crash) vs manual commit (commit only AFTER successful processing - stronger guarantee)

SPRING KAFKA: `KafkaTemplate.send()` (producing) + `@KafkaListener` (consuming) - same "annotate, Spring handles plumbing" pattern as `@Scheduled/@Async/@Transactional/@Cacheable`

7. Homework (Covers Entire Kafka Section)

- 1 Run Kafka locally via Docker, create a topic with 3 partitions, and use the console producer/consumer to send and receive messages manually.
- 2 Write a plain Java `KafkaProducer` and `KafkaConsumer`, observing which partition each message lands in based on its key.
- 3 Run TWO instances of the same consumer group and observe how Kafka splits partition assignment between them; then stop one and observe the automatic rebalance.

- 4 Convert your manual producer/consumer into Spring Kafka's `KafkaTemplate@KafkaListener`, confirming identical behavior with dramatically less code.
-

FINAL SUMMARY — KAFKA COMPLETE

This concludes the **Kafka** section — 8 topics:

- 1 **Brokers** — the persistent, distributed alternative to Redis Pub/Sub.
- 2 **Topics** — named message feeds.
- 3 **Partitions** — the unit of parallelism, and the "ordering only within a partition" rule.
- 4 **Producers** — writing messages, the `acks` durability trade-off.
- 5 **Consumers** — reading messages, tracking position.
- 6 **Consumer Groups** — parallel processing within a group, broadcast across groups.
- 7 **Offsets** — auto vs manual commit, and the crash-safety trade-off.
- 8 **Spring Kafka Integration** — `KafkaTemplate@KafkaListener` eliminating raw client boilerplate.

Kafka is the backbone of event-driven microservice architectures — the direct foundation for the Microservices section ahead, where services communicate asynchronously through exactly these topics and consumer groups instead of only direct synchronous REST calls.

(END OF KAFKA MASTER NOTES — ready to proceed to Microservices whenever you say "Next Topic.")